
boolean



JS - Timing functions

Timing functions

Javascript conosce il tempo



Tra le funzioni che Javascript offre, ce ne sono due che permettono di eseguire funzioni tenendo conto del passare del tempo.

Queste funzioni ci offrono il primo approccio alla **programmazione asincrona**.

setTimeout

Facciamo qualcosa **dopo un tot**

1 **setTimeout** attiva una funzione **dopo un certo tempo**.

setTimeout(nomeFunzione, tempoDaAspettare);

1. invoca una funzione
2. che si attiverà quando saranno passati tot millisecondi

```
1   setTimeout(myFunction, 3000);
2
3   function myFunction() {
4     alert('Hello');
5   }
```

setInterval

Facciamo qualcosa **ogni tot**

2 **setInterval** attiva una funzione **all'infinito, ogni tot di tempo**.

setInterval(nomeFunzione, tempoPerOgniRipetizione);

1. invoca una funzione
2. che si attiverà ogni tot millisecondi

```
1   const clock = setInterval(myFunction, 3000);
2
3   function myFunction() {
4      alert('Hello');
5   }
```

setInterval

Facciamo qualcosa **ogni tot**

Come fermo un setInterval?

1. Salvo il risultato del **setInterval** in una **variabile**
2. La passo alla funzione:
   clearInterval(nomeVariabileSetInterval)

```
1   const clock = setInterval(myFunction, 3000);
2   //console.log(clock) <- cosa ci restituirà?
3
4   function myFunction() {
5    alert('Hello');
6   }
7
8   clearInterval(clock);
```

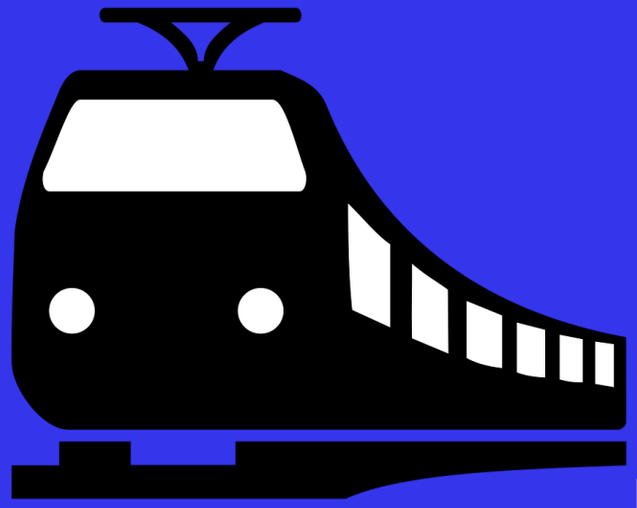


Per terminare l'esecuzione di **setTimeout()** si usa **clearTimeout**.

Ore 10.00

“Attenzione al Binario 3 è in arrivo
il treno delle ore 8.00!”

Asincronicità



Timing functions / Async

Javascript può gestire la priorità delle funzioni da eseguire (grazie al browser)

Cos'è la **programmazione Sincrona**?

Fino ad ora abbiamo sempre programmato in **maniera Sincrona**:

ad ogni riga di codice corrisponde un'azione nel tempo e finché tale azione non è conclusa il programma non va avanti.

```
1 const array = [1, 2, 3, 4];
2
3 for (let i = 0; i < array.length; i++) {
4  console.log(array[i]); //stampa un elemento alla volta
5 }
6
7 console.log('Ciao');//viene stampato dopo il for
```

Timing functions / Async

Javascript può gestire la priorità delle funzioni da eseguire (grazie al browser)

Cos'è la **programmazione Asincrona**?

La **programmazione Asincrona** consente di definire un'azione che avverrà in un altro momento.

```
1 setTimeout(myFunction, 3000); //stampa dopo 3 secondi
2
3 console.log('Ciao');//viene stampato per primo
4
5
6 function myFunction() {
7    console.log('Ciao Asincrono');
8 }
```

Timing functions / Async

Javascript può gestire la priorità delle funzioni da eseguire (grazie al browser)

Ma se abbiamo più eventi chi decide le priorità?
In che modo vengono gestiti gli eventi asincroni?

Vediamo i vari step in azione!

```
1  setTimeout(myFunction, 4000);
2
3  console.log('Ciao');
4  console.log('Ancora un evento sincrono');
5
6
7
8  function myFunction() {
9     console.log('Ciao Asincrono');
10 }
```

Timing functions / Async

Vediamo insieme un esempio pratico, parcheggiamo un'auto!

```
inizio un contatore che dopo quattro secondi
avvierà la funzione esciDalParcheggio()

faccio passare la prima auto (nera)

faccio passare la seconda auto (rossa)

preparo una funzione per far passare anche la
mia auto quando necessario

definisco una funzione che espliciti in
console il passaggio di una data auto
```

```
1  setTimeout(esciDalParcheggio, 4000);
2
3  passaAuto('Macchina nera');
4  passaAuto('Macchina rossa');
5
6  function esciDalParcheggio() {
7     console.log('Esco dal parcheggio!');
8  }
9
10 function passaAuto(nomeAuto){
11   console.log("Sta passando la " + nomeAuto);
12   // codice che fa passare l'automobile nomeAuto
13   console.log("È passata la " + nomeAuto);
14 }
```

Timing functions / Async

Javascript può gestire la priorità delle funzioni da eseguire (grazie al browser)

1 Il browser legge la nostra funzione setTimeout e la richiama.



"Vorrei parcheggiare la mia auto per un'ora, al termine dei quali vorrei uscire dal parcheggio"

```
1  setTimeout(esciDalParcheggio, 3600000);
```

Un'ora sono 60 minuti, ognuno dei quali sono 60 secondi, ovvero 3600 secondi.

Dobbiamo però convertire in millisecondi moltiplicando per mille: il totale sarà 3.600.000ms.

Timing functions / Async

Javascript può gestire la priorità delle funzioni da eseguire (grazie al browser)

2 Il browser:

- valuta la funzione che deve essere richiamata dal setTimeout e la mette in attesa
- avvia il passaAuto()



```
1 setTimeout(esciDalParcheggio, 3600000);
2
3 passaAuto("Macchina nera");
4 //  Sta passando la Macchina Nera
5 //  È passata la Macchina Nera
```

Nel frattempo passano altre vetture: prima ***la macchina nera***

Timing functions / Async

Javascript può gestire la priorità delle funzioni da eseguire (grazie al browser)

3 Il browser:

- Avvia il secondo passaAuto()



```
1 setTimeout(esciDalParcheggio, 3600000);
2
3 passaAuto("Macchina nera");
4 //  Sta passando la Macchina Nera
5 //  È passata la Macchina Nera
6
7 passaAuto('Macchina Rossa');
8 //  Sta passando la Macchina Rossa
9 //  È passata la Macchina Rossa
```

E poi passerà anche la **macchina rossa!**

Timing functions / Async

Javascript può gestire la priorità delle funzioni da eseguire (grazie al browser)

4 Il browser:

- Passata un'ora mette la funzione esciDalParcheggio() nella lista delle callback da avviare



"Mi preparo ad uscire, dando sempre la precedenza a tutte le auto che stanno già passando"

```
1 setTimeout(esciDalParcheggio, 3600000);
2
3 passaAuto("Macchina nera");
4 passaAuto('Macchina Rossa');
5
6 function esciDalParcheggio(){
7    passaAuto('Mia auto');
8 }
```

Timing functions / Async

Javascript può gestire la priorità delle funzioni da eseguire (grazie al browser)

4 Il browser:

- Passata un'ora mette la funzione esciDalParcheggio() nella lista delle callback da avviare



"Mi preparo ad uscire, dando sempre la precedenza a tutte le auto che stanno già passando"

```
1 setTimeout(esciDalParcheggio, 3600000);
2
3 passaAuto("Macchina nera");
4 passaAuto('Macchina Rossa');
5
6 function esciDalParcheggio(){
7    passaAuto('Mia auto');
8 }
```

Timing functions / Async

Javascript può gestire la priorità delle funzioni da eseguire (grazie al browser)

4 Il browser:

- Passata un'ora mette la funzione esciDalParcheggio() nella lista delle callback da avviare



"Mi preparo ad uscire, dando sempre la precedenza a tutte le auto che stanno già passando"

```
1 setTimeout(esciDalParcheggio, 3600000);
2
3 passaAuto("Macchina nera");
4 passaAuto('Macchina Rossa');
5
6 function esciDalParcheggio(){
7    passaAuto('Mia auto');
8 }
```

Timing functions / Async

Javascript può gestire la priorità delle funzioni da eseguire (grazie al browser)

5 Il browser:

- Avvia la funzione esciDalParcheggio() e di conseguenza il console.log()



"Finalmente sono chiamato ad uscire dal parcheggio, dopo aver dato la precedenza a tutte le vetture chiamate a passare prima di me"

```
1 setTimeout(esciDalParcheggio, 3600000);
2
3 passaAuto("Macchina nera");
4 passaAuto('Macchina Rossa');
5
6 function esciDalParcheggio(){
7    passaAuto('Mia auto');
8 }
```

Timing functions / Async

Quindi... ricapitolando

```
setTimeout(esciDalParcheggio, 3600000);

passaAuto("Macchina nera");
//  Sta passando la Macchina Nera
//  È passata la Macchina Nera

passaAuto('Macchina Rossa');
//  Sta passando la Macchina Rossa
//  È passata la Macchina Rossa

function esciDalParcheggio(){
  passaAuto('Mia auto');
  //  Sta passando la Mia auto
  //  È passata la Mia auto
}
```

Se la macchina nera e la macchina rossa dovessero **richiedere due ore** per passare lungo una strada particolarmente trafficata, **la mia automobile quando passerà**?

Solo una volta che:

- Il tempo di timeout sia trascorso

- Tutte le funzioni chiamate precendentemente abbiano concluso la loro esecuzione

Anonymous functions

Quando si deve passare una funzione come argomento, è possibile utilizzare una **funzione anonima** (*anonymous function*) cioè una funzione senza nome.

```
1  const clock = setInterval(myFunction, 4000);
2
3
4  // funzione con nome
5  function myFunction() {
6    alert('Hello');
7  }
8
9  // clearInterval(clock)
```

```
1  const clock = setInterval(function () { // funzione anonima
2
3    alert('Hello');
4
5  }, 4000);
6
7
8  // clearInterval(clock)
```



LIVE CODING

ESERCITAZIONE